Lecture 6. Basics of Functional Programming in Haskell

Programming Paradigms, Fall 2026 — 30 Sep 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. Where We Are
2. A Module and GHCi
3. Types
4. Functions
5. Lists
6. Types of Our Own

Where We Are

This Lecture

Last week: closures, and the functional style in Python and JavaScript.
Types were checked while the program ran.

This Lecture



Last week: closures, and the functional style in Python and JavaScript.
Types were checked while the program ran.

This week: Haskell.
Types are checked **before** the program runs.
`map`, `filter`, recursion, and immutable values come with us.

This Lecture

Last week: closures, and the functional style in Python and JavaScript.
Types were checked while the program ran.

This week: Haskell.
Types are checked **before** the program runs.
`map`, `filter`, recursion, and immutable values come with us.

Three things to get out of it:

1. **Reading** a definition by equations, patterns, and guards.
2. **Writing** a function over a list by recursion, with its type.
3. **Reading** the compiler's answer when a type does not fit.

This Lecture

Last week: closures, and the functional style in Python and JavaScript.
Types were checked while the program ran.

This week: Haskell.
Types are checked **before** the program runs.
`map`, `filter`, recursion, and immutable values come with us.

Three things to get out of it:

1. **Reading** a definition by equations, patterns, and guards.
2. **Writing** a function over a list by recursion, with its type.
3. **Reading** the compiler's answer when a type does not fit.

After today you can write a module of small functions and check it in GHCi.

Quiz 5: A Closure Captures a Binding

Many of you asked why `cheap(80)` changed after the second assignment:

Python

```
limit = 100
cheap = lambda price: price < limit
limit = 50
cheap(80)      # False
```

Quiz 5: A Closure Captures a Binding

Many of you asked why `cheap(80)` changed after the second assignment:

Python

```
limit = 100
cheap = lambda price: price < limit
limit = 50
cheap(80)      # False
```

The closure holds the *binding* of `limit`, and reads it when called.
The assignment changed what the binding holds.

Quiz 5: A Closure Captures a Binding

Many of you asked why `cheap(80)` changed after the second assignment:

Python

```
limit = 100
cheap = lambda price: price < limit
limit = 50
cheap(80)      # False
```

The closure holds the *binding* of `limit`, and reads it when called.
The assignment changed what the binding holds.

A loop variable is one binding, assigned three times.

Quiz 5: A Closure Captures a Binding

Many of you asked why `cheap(80)` changed after the second assignment:

Python

```python
limit = 100
cheap = lambda price: price < limit
limit = 50
cheap(80)      # False
```

The closure holds the *binding* of `limit`, and reads it when called.
The assignment changed what the binding holds.

A loop variable is one binding, assigned three times.

Haskell

```haskell
limit = 100
cheap price = price < limit      -- a second limit = 50 is refused
```

Quiz 5: A Collection, an Iterator, and a Haskell List

Some of you asked what `iter(value)` is for, and which of the three to prefer:

```
values = [10, 20, 30]   # a collection: stored, walked again and again
it = iter(values)       # an iterator: one pass over it, with a position
next(it)                # 10, and the position moves on
```

Quiz 5: A Collection, an Iterator, and a Haskell List

Some of you asked what `iter(value)` is for, and which of the three to prefer:

```
values = [10, 20, 30]   # a collection: stored, walked again and again
it = iter(values)       # an iterator: one pass over it, with a position
next(it)                # 10, and the position moves on
```

A generator is an iterator that computes its values as it goes.

Quiz 5: A Collection, an Iterator, and a Haskell List

Some of you asked what `iter(value)` is for, and which of the three to prefer:

```
values = [10, 20, 30]   # a collection: stored, walked again and again
it = iter(values)       # an iterator: one pass over it, with a position
next(it)                # 10, and the position moves on
```

A generator is an iterator that computes its values as it goes.

Store a collection when the values are needed twice.
Use an iterator or a generator when they are read once, or may have no end.

Quiz 5: A Collection, an Iterator, and a Haskell List

Some of you asked what `iter(value)` is for, and which of the three to prefer:

```
values = [10, 20, 30]   # a collection: stored, walked again and again
it = iter(values)       # an iterator: one pass over it, with a position
next(it)                # 10, and the position moves on
```

A generator is an iterator that computes its values as it goes.

Store a collection when the values are needed twice.
Use an iterator or a generator when they are read once, or may have no end.

A Haskell list is a value. There is nothing to use up.
Values on demand come back in topic 9, and they are lists too.

What Haskell Is

A language designed in 1990 by a committee, to give the research on lazy functional languages one common tongue. Named after Haskell Curry.

What Haskell Is

A language designed in 1990 by a committee, to give the research on lazy functional languages one common tongue. Named after Haskell Curry.

- **Purely functional**: a function computes a value and does nothing else.
- **Statically typed**, with the types mostly inferred, so we write few of them.
- **Lazy**: a value is computed when something asks for it.
- Compiled to native code by GHC, with a REPL and a package ecosystem.

What Haskell Is



A language designed in 1990 by a committee, to give the research on lazy functional languages one common tongue. Named after Haskell Curry.

- **Purely functional**: a function computes a value and does nothing else.
- **Statically typed**, with the types mostly inferred, so we write few of them.
- **Lazy**: a value is computed when something asks for it.
- Compiled to native code by GHC, with a REPL and a package ecosystem.

Ideas tend to arrive here first and spread: type classes inspired Rust's traits, `Maybe` has cousins in Rust, Swift and Java, and QuickCheck's property testing is in every language now.

Where Haskell Is Used



- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.

Where Haskell Is Used

- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.
- **Web**: the back end of Sirius Courses, the online school of the Sirius centre, is in Haskell.

Where Haskell Is Used

- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.
- **Web**: the back end of Sirius Courses, the online school of the Sirius centre, is in Haskell.
- **Infrastructure**: Meta's anti-abuse system Sigma, and the Cardano blockchain.

Where Haskell Is Used



- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.
- **Web**: the back end of Sirius Courses, the online school of the Sirius centre, is in Haskell.
- **Infrastructure**: Meta's anti-abuse system Sigma, and the Cardano blockchain.
- **Tools you may already use**: Pandoc converts documents, ShellCheck lints shell scripts, PostgREST serves a database as an API.

Where Haskell Is Used



- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.
- **Web**: the back end of Sirius Courses, the online school of the Sirius centre, is in Haskell.
- **Infrastructure**: Meta's anti-abuse system Sigma, and the Cardano blockchain.
- **Tools you may already use**: Pandoc converts documents, ShellCheck lints shell scripts, PostgREST serves a database as an API.
- **Compilers and proof assistants**: GHC itself, and Agda.

Where Haskell Is Used



- **Finance**: Standard Chartered runs its trading platform on a Haskell dialect; Mercury, a bank, is written in it.
- **Web**: the back end of Sirius Courses, the online school of the Sirius centre, is in Haskell.
- **Infrastructure**: Meta's anti-abuse system Sigma, and the Cardano blockchain.
- **Tools you may already use**: Pandoc converts documents, ShellCheck lints shell scripts, PostgREST serves a database as an API.
- **Compilers and proof assistants**: GHC itself, and Agda.

It is a small community, and a demanding language to learn.
What it gives in return carries over to any language:
the habit of designing first, and writing the design down in the types.

The Same Mistake in Four Languages

We add one to a string, at the prompt of each language.
What happens?

Racket

```
> (+ "4" 1)
```

Python

```
>>> "4" + 1
```

JavaScript

```
> "4" + 1
```

Haskell

```
ghci> "4" + 1
```

The Same Mistake in Four Languages

We add one to a string, at the prompt of each language.
What happens?

Racket

```
> (+ "4" 1)
+: contract violation
  expected: number?
  given: "4"
```

JavaScript

```
> "4" + 1
'41'
```

Python

```
>>> "4" + 1
TypeError: can only concatenate
  str (not "int") to str
```

Haskell

```
ghci> "4" + 1
error: [GHC-39999]
  • No instance for ‘Num String’
    arising from a use of ‘+’
```

The Same Mistake in Four Languages

We add one to a string, at the prompt of each language.
What happens?

Racket

```
> (+ "4" 1)
+: contract violation
  expected: number?
  given: "4"
```

Python

```
>>> "4" + 1
TypeError: can only concatenate
  str (not "int") to str
```

JavaScript

```
> "4" + 1
'41'
```

Haskell

```
ghci> "4" + 1
error: [GHC-39999]
  • No instance for ‘Num String’
    arising from a use of ‘+’
```

Racket and Python fail when the line runs, and JavaScript never does.

The Same Mistake in Four Languages

We add one to a string, at the prompt of each language.
What happens?

Racket

```
>  (+ "4" 1)
+:  contract violation
  expected:  number?
  given:  "4"
```

JavaScript

```
>  "4" + 1
'41'
```

Python

```
»>  "4" + 1
TypeError:  can only concatenate
  str (not "int") to str
```

Haskell

```
ghci>  "4" + 1
error:  [GHC-39999]
  • No instance for ‘Num String’
    arising from a use of ‘+’
```

Racket and Python fail when the line runs, and JavaScript never does.

Haskell refuses the program before it runs.

A Module and GHCi

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$ ghci Parcels.hs
```

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$  ghci Parcels.hs
ghci>  heavy
```

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$ ghci Parcels.hs
ghci> heavy
True
```

GHCi loads the file and evaluates expressions at the prompt.

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$  ghci Parcels.hs
ghci>  heavy
True
ghci>  :type weight
```

GHCi loads the file and evaluates expressions at the prompt.

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$ ghci Parcels.hs
ghci> heavy
True
ghci> :type weight
weight :: Double
```

GHCi loads the file and evaluates expressions at the prompt.

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$ ghci Parcels.hs
ghci> heavy
True
ghci> :type weight
weight :: Double
ghci> :reload
```

GHCi loads the file and evaluates expressions at the prompt.

`:reload` reads the file again after an edit.

A First Module

A Haskell program is a file of **declarations**:

Parcels.hs

```
module Parcels where

limit :: Double
limit = 2000
weight :: Double
weight = 5400
heavy :: Bool
heavy = weight > limit
```

in a terminal, in the same directory

```
$ ghci Parcels.hs
ghci> heavy
True
ghci> :type weight
weight :: Double
ghci> :reload
```

GHCi loads the file and evaluates expressions at the prompt.

`:reload` reads the file again after an edit.

The problem sets work this way: the answer file has to load.

Declarations, Not Assignments

The sign `=` says that the two sides are equal, once and for all:

```
limit = 2000
limit = 5000
```

Declarations, Not Assignments

The sign `=` says that the two sides are equal, once and for all:

```
limit = 2000
limit = 5000
```

Question

What is `limit` now?

Declarations, Not Assignments

The sign `=` says that the two sides are equal, once and for all:

```
limit = 2000
limit = 5000
```

Question

What is `limit` now?

```
Parcels.hs:7:1: error: [GHC-29916]
    Multiple declarations of ‘limit’
    Declared at: Parcels.hs:6:1
                 Parcels.hs:7:1
```

Neither. A name is declared once.

Declarations, Not Assignments

The sign `=` says that the two sides are equal, once and for all:

```
limit = 2000
limit = 5000
```

Question

What is `limit` now?

```
Parcels.hs:7:1: error: [GHC-29916]
    Multiple declarations of ‘limit’
    Declared at: Parcels.hs:6:1
                 Parcels.hs:7:1
```

Neither. A name is declared once.

Nothing is ever updated, so the order of the declarations does not matter.

Type Signatures

A **type signature** states the type of a declaration:

```
limit :: Double
limit = 2000
```

Read `limit :: Double` as “`limit` has type `Double`”.

[1] Luis Damas and Robin Milner (1982). **“Principal type-schemes for functional programs”.** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. Albuquerque, New Mexico: Association for Computing Machinery. URL: https://doi.org/10.1145/582153.582176.

Type Signatures

A **type signature** states the type of a declaration:

```
limit :: Double
limit = 2000
```

Read `limit :: Double` as “`limit` has type `Double`”.

GHC infers most types on its own[1].
We write the signature of every top-level definition anyway.

[1] Luis Damas and Robin Milner (1982). **“Principal type-schemes for functional programs”.** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. Albuquerque, New Mexico: Association for Computing Machinery. URL: https://doi.org/10.1145/582153.582176.

Type Signatures

A **type signature** states the type of a declaration:

```
limit :: Double
limit = 2000
```

Read `limit :: Double` as "`limit` has type `Double`".

GHC infers most types on its own[1].
We write the signature of every top-level definition anyway.

The signature says what we meant.
If the definition disagrees, the error points at the definition.

[1] Luis Damas and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. Albuquerque, New Mexico: Association for Computing Machinery. URL: https://doi.org/10.1145/582153.582176.

Functions and Their Types

A function is an equation with its arguments on the left:

```
withNds :: Double -> Double
withNds price = price * 1.2

insured :: Double -> Double -> Double
insured value price = price + 0.01 * value
```

Functions and Their Types

A function is an equation with its arguments on the left:

```
withNds :: Double -> Double
withNds price = price * 1.2

insured :: Double -> Double -> Double
insured value price = price + 0.01 * value
```

In the type, an arrow separates each argument from what follows it.

Functions and Their Types

A function is an equation with its arguments on the left:

```haskell
withNds :: Double -> Double
withNds price = price * 1.2

insured :: Double -> Double -> Double
insured value price = price + 0.01 * value
```

In the type, an arrow separates each argument from what follows it.

Thus, `insured` takes two **`Double`**s, and then gives a **`Double`**.

Calls

The arguments follow the function, with nothing in between:

```
insured 3000 400              -- 430.0
withNds price * 2             -- (withNds price) * 2
withNds (insured 3000 400)    -- a nested call needs parentheses
```

Calls

The arguments follow the function, with nothing in between:

```
insured 3000 400              -- 430.0
withNds price * 2             -- (withNds price) * 2
withNds (insured 3000 400)    -- a nested call needs parentheses
```

A call binds tighter than any operator.

Calls

The arguments follow the function, with nothing in between:

```
insured 3000 400              -- 430.0
withNds price * 2             -- (withNds price) * 2
withNds (insured 3000 400)    -- a nested call needs parentheses
```

A call binds tighter than any operator.

In Racket every call had parentheses.
Here only the nested one does.

Types

Numbers

Three types of numbers are enough for this course:

```haskell
count :: Int            -- a machine integer, 64 bits
count = 42
big :: Integer          -- an integer of any size
big = 2 ^ 64            -- 18446744073709551616
weight :: Double        -- a floating-point number
weight = 1.5
```

Numbers

Three types of numbers are enough for this course:

```
count :: Int            -- a machine integer, 64 bits
count = 42
big :: Integer          -- an integer of any size
big = 2 ^ 64            -- 18446744073709551616
weight :: Double        -- a floating-point number
weight = 1.5
```

The largest `Int` is `maxBound`, that is, `9223372036854775807`.

What is `maxBound + 1`?

Numbers

Three types of numbers are enough for this course:

```haskell
count :: Int            -- a machine integer, 64 bits
count = 42
big :: Integer          -- an integer of any size
big = 2 ^ 64            -- 18446744073709551616
weight :: Double        -- a floating-point number
weight = 1.5
```

The largest `Int` is `maxBound`, that is, `9223372036854775807`.

What is `maxBound + 1`?

`-9223372036854775808`: an `Int` wraps around.

Numbers

Three types of numbers are enough for this course:

```
count :: Int            -- a machine integer, 64 bits
count = 42
big :: Integer          -- an integer of any size
big = 2 ^ 64            -- 18446744073709551616
weight :: Double        -- a floating-point number
weight = 1.5
```

The largest `Int` is `maxBound`, that is, `9223372036854775807`.
What is `maxBound + 1`?

`-9223372036854775808`: an `Int` wraps around.

A literal takes the number type its context asks for: `weight = 5` is `5.0`.

Reading a Signature With a Constraint

GHCi shows the type of addition as

```
(+) :: Num a => a -> a -> a
```

Reading a Signature With a Constraint

GHCi shows the type of addition as

```
(+) :: Num a => a -> a -> a
```

`a` is a **type variable**: it stands for any type.

Reading a Signature With a Constraint

GHCi shows the type of addition as

```
(+) :: Num a => a -> a -> a
```

`a` is a **type variable**: it stands for any type.

`Num a` is a **constraint**: `a` is a number type.

Reading a Signature With a Constraint

GHCi shows the type of addition as

```
(+) :: Num a => a -> a -> a
```

`a` is a **type variable**: it stands for any type.

`Num a` is a **constraint**: `a` is a number type.

Both arguments have the *same* type `a`, and so does the result.

Reading a Signature With a Constraint

GHCi shows the type of addition as

```
(+) :: Num a => a -> a -> a
```

`a` is a **type variable**: it stands for any type.

`Num a` is a **constraint**: `a` is a number type.

Both arguments have the *same* type `a`, and so does the result.

`Num` is a **type class**: a group of operations that a type may provide.
Next week we look inside one.

Adding an `Int` to a `Double`

`count :: Int` and `weight :: Double`. What does this produce?

```
bad = count + weight
```

Adding an `Int` to a `Double`

`count :: Int` and `weight :: Double`. What does this produce?

```
bad = count + weight
```

```
Parcels.hs:14:15: error: [GHC-83865]
    • Couldn't match expected type ‘Int’ with actual type ‘Double’
    • In the second argument of ‘(+)’, namely ‘weight’
      In the expression: count + weight
      In an equation for ‘bad’: bad = count + weight
```

An error, before anything runs.
The first argument fixed `a` to **`Int`**.
The second was *expected* to be an **`Int`** too, and it is *actually* a **`Double`**.

Reading an Error

Every type error from GHC has the same three parts:

Reading an Error

Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.

Reading an Error

Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.
2. **What**: the type *expected* there, and the type *actually* found.

Reading an Error

Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.
2. **What**: the type *expected* there, and the type *actually* found.
3. **In what**: the expression, the larger expression, the equation.

Reading an Error

Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.
2. **What**: the type *expected* there, and the type *actually* found.
3. **In what**: the expression, the larger expression, the equation.

Read the first two, then the innermost expression of the third.

Reading an Error



Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.
2. **What**: the type *expected* there, and the type *actually* found.
3. **In what**: the expression, the larger expression, the equation.

Read the first two, then the innermost expression of the third.

The code in brackets, such as `[GHC-83865]`, is a page at `https://errors.haskell.org/`.

Reading an Error



Every type error from GHC has the same three parts:

1. **Where**: file, line, column, and the code with the place marked.
2. **What**: the type *expected* there, and the type *actually* found.
3. **In what**: the expression, the larger expression, the equation.

Read the first two, then the innermost expression of the third.

The code in brackets, such as `[GHC-83865]`, is a page at `https://errors.haskell.org/`.

GHC reads left to right, so the mismatch is reported at the second argument.

Converting on Purpose



```haskell
perParcel :: Double
perParcel = weight / fromIntegral count      -- 1.5 / 42.0
```

Converting on Purpose

```
perParcel :: Double
perParcel = weight / fromIntegral count      -- 1.5 / 42.0
```

`fromIntegral` converts an integer to the number type asked for.

`round`, `floor` and `ceiling` go the other way.

Converting on Purpose



```haskell
perParcel :: Double
perParcel = weight / fromIntegral count      -- 1.5 / 42.0
```

`fromIntegral` converts an integer to the number type asked for.
`round`, `floor` and `ceiling` go the other way.

```haskell
7 / 2           -- 3.5,  division of Doubles
7 `div` 2       -- 3,    integer division
7 `mod` 2       -- 1,    the remainder
2 ^ 10          -- 1024, a power with an integer exponent
```

`/` is for **`Double`**, `div` for **`Int`**. Backticks make a function infix.

Booleans, Characters, and Strings



```haskell
heavy :: Bool
heavy = weight > 2000 && not fragile      -- also ||, ==, /=
initial :: Char
initial = 'B'                              -- one character
name :: String
name = "books"                             -- a string
```

Booleans, Characters, and Strings



```haskell
heavy :: Bool
heavy = weight > 2000 && not fragile      -- also ||, ==, /=
initial :: Char
initial = 'B'                              -- one character
name :: String
name = "books"                             -- a string
```

Comparisons give a **`Bool`**.
Single quotes make a **`Char`**, double quotes a **`String`**.

Booleans, Characters, and Strings



```haskell
heavy :: Bool
heavy = weight > 2000 && not fragile     -- also ||, ==, /=
initial :: Char
initial = 'B'                             -- one character
name :: String
name = "books"                            -- a string
```

Comparisons give a **`Bool`**.
Single quotes make a **`Char`**, double quotes a **`String`**.

```haskell
name ++ ": " ++ show weight      -- "books: 1.5"
```

`++` joins strings, and `show` writes any value as one.

Booleans, Characters, and Strings



```haskell
heavy :: Bool
heavy = weight > 2000 && not fragile      -- also ||, ==, /=
initial :: Char
initial = 'B'                              -- one character
name :: String
name = "books"                             -- a string
```

Comparisons give a **`Bool`**.
Single quotes make a **`Char`**, double quotes a **`String`**.

```haskell
name ++ ": " ++ show weight      -- "books: 1.5"
```

`++` joins strings, and `show` writes any value as one.

All of this comes from the **Prelude**, Haskell's standard module.

Functions

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

What is `zone "DE"`?

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

What is `zone "DE"`? `3`: the underscore matches anything.

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

What is `zone "DE"`? `3`: the underscore matches anything.

What if the last equation came first?

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

What is `zone "DE"`? `3`: the underscore matches anything.

What if the last equation came first? Every country would be in zone 3.

Equations and Patterns

A function may have several equations, tried from the top:

```
zone :: String -> Int
zone "RU" = 1
zone "KZ" = 2
zone "BY" = 2
zone _    = 3
```

A literal is a **pattern**: the equation applies when the argument equals it.

What is `zone "DE"`? `3`: the underscore matches anything.

What if the last equation came first? Every country would be in zone 3.

In Racket this was a **`cond`** with an `equal?` in each clause.

Tuples

A **tuple** holds a fixed number of values, each of its own type:

```
parcel :: (String, Double)
parcel = ("books", 5400)

label :: (String, Double) -> String
label (name, w) = name ++ ": " ++ show w ++ " g"
```

Tuples

A **tuple** holds a fixed number of values, each of its own type:

```
parcel :: (String, Double)
parcel = ("books", 5400)

label :: (String, Double) -> String
label (name, w) = name ++ ": " ++ show w ++ " g"
```

One argument, the pair.

The pattern `(name, w)` names both parts.

Tuples

A **tuple** holds a fixed number of values, each of its own type:

```
parcel :: (String, Double)
parcel = ("books", 5400)

label :: (String, Double) -> String
label (name, w) = name ++ ": " ++ show w ++ " g"
```

One argument, the pair.
The pattern `(name, w)` names both parts.

The Prelude has `fst` and `snd`. A pattern reads better.

Tuples

A **tuple** holds a fixed number of values, each of its own type:

```
parcel :: (String, Double)
parcel = ("books", 5400)

label :: (String, Double) -> String
label (name, w) = name ++ ": " ++ show w ++ " g"
```

One argument, the pair.
The pattern `(name, w)` names both parts.

The Prelude has `fst` and `snd`. A pattern reads better.

Can `label` take `("books", 5400, True)`?

Tuples

A **tuple** holds a fixed number of values, each of its own type:

```
parcel :: (String, Double)
parcel = ("books", 5400)

label :: (String, Double) -> String
label (name, w) = name ++ ": " ++ show w ++ " g"
```

One argument, the pair.
The pattern `(name, w)` names both parts.

The Prelude has `fst` and `snd`. A pattern reads better.

Can `label` take `("books", 5400, True)`?
No: a triple is a different type.

Guards

A **guard** adds a condition to an equation:

$$postage(w) = \begin{cases} 250 & \text{if } w \leq 500 \\ 400 & \text{if } 500 < w \leq 2000 \\ 900 & \text{otherwise} \end{cases}$$

```
postage :: Double -> Double
postage w
  | w <= 500  = 250
  | w <= 2000 = 400
  | otherwise = 900
```

Guards

A **guard** adds a condition to an equation:

$$postage(w) = \begin{cases} 250 & \text{if } w \leq 500 \\ 400 & \text{if } 500 < w \leq 2000 \\ 900 & \text{otherwise} \end{cases}$$

```
postage :: Double -> Double
postage w
  | w <= 500  = 250
  | w <= 2000 = 400
  | otherwise = 900
```

What is `postage 500`?

Guards

A **guard** adds a condition to an equation:

$$postage(w) = \begin{cases} 250 & \text{if } w \leq 500 \\ 400 & \text{if } 500 < w \leq 2000 \\ 900 & \text{otherwise} \end{cases}$$

```
postage :: Double -> Double
postage w
  | w <= 500  = 250
  | w <= 2000 = 400
  | otherwise = 900
```

What is `postage 500`?
`250`: the first true guard wins.

Guards

A **guard** adds a condition to an equation:

$$postage(w) = \begin{cases} 250 & \text{if } w \leq 500 \\ 400 & \text{if } 500 < w \leq 2000 \\ 900 & \text{otherwise} \end{cases}$$

```
postage :: Double -> Double
postage w
  | w <= 500  = 250
  | w <= 2000 = 400
  | otherwise = 900
```

What is `postage 500`?
`250`: the first true guard wins.

`otherwise` is another name for **`True`**.
The second guard only sees weights above 500, so it needs no lower bound.

The same function, as an expression:

```haskell
postage :: Double -> Double
postage w =
  if w <= 500 then 250
  else if w <= 2000 then 400
  else 900
```

The same function, as an expression:

```
postage :: Double -> Double
postage w =
  if w <= 500 then 250
  else if w <= 2000 then 400
  else 900
```

An `if` is an expression, so it always has an `else`.
Without one it is a syntax error.

The same function, as an expression:

```
postage :: Double -> Double
postage w =
  if w <= 500 then 250
  else if w <= 2000 then 400
  else 900
```

An `if` is an expression, so it always has an `else`.
Without one it is a syntax error.

An expression can stand wherever a value can.
Guards cannot.

The same function, as an expression:

```
postage :: Double -> Double
postage w =
  if w <= 500 then 250
  else if w <= 2000 then 400
  else 900
```

An `if` is an expression, so it always has an `else`.
Without one it is a syntax error.

An expression can stand wherever a value can.
Guards cannot.

For a chain of conditions, the guards read better.

A **case** expression matches patterns inside an expression:

```
describe :: (String, Double) -> String
describe p =
  case p of
    (name, w) | w > 2000 -> name ++ " is heavy"
    (name, _)            -> name ++ " is light"
```

A **case** expression matches patterns inside an expression:

```haskell
describe :: (String, Double) -> String
describe p =
  case p of
    (name, w) | w > 2000 -> name ++ " is heavy"
    (name, _)            -> name ++ " is light"
```

Each alternative: a pattern, a guard if needed, an arrow, a value.
They are tried from the top.

A **case** expression matches patterns inside an expression:

```
describe :: (String, Double) -> String
describe p =
  case p of
    (name, w) | w > 2000 -> name ++ " is heavy"
    (name, _)            -> name ++ " is light"
```

Each alternative: a pattern, a guard if needed, an arrow, a value.
They are tried from the top.

```
describe (name, w)
  | w > 2000  = name ++ " is heavy"
  | otherwise = name ++ " is light"
```

Equations are a **case** on the arguments, with a nicer syntax.

Three Ways to Choose

	guards	**`if`**	**`case`**
tests	conditions	one condition	patterns, and then conditions
stands	after a pattern	in an expression	in an expression
reads best for	a chain of conditions	one choice, inside a value	a choice by shape
Racket	`cond`	`if`	`cond` with `empty?`, `first`

Three Ways to Choose

	guards	**`if`**	**`case`**
tests	conditions	one condition	patterns, and then conditions
stands	after a pattern	in an expression	in an expression
reads best for	a chain of conditions	one choice, inside a value	a choice by shape
Racket	`cond`	`if`	`cond` with `empty?`, `first`

A guard follows a pattern: on an equation, or on a **`case`** alternative.

Three Ways to Choose

	guards	**`if`**	**`case`**
tests	conditions	one condition	patterns, and then conditions
stands	after a pattern	in an expression	in an expression
reads best for	a chain of conditions	one choice, inside a value	a choice by shape
Racket	`cond`	`if`	`cond` with `empty?`, `first`

A guard follows a pattern: on an equation, or on a **`case`** alternative.

Each can be rewritten as any other.

Three Ways to Choose

	guards	**`if`**	**`case`**
tests	conditions	one condition	patterns, and then conditions
stands	after a pattern	in an expression	in an expression
reads best for	a chain of conditions	one choice, inside a value	a choice by shape
Racket	`cond`	`if`	`cond` with `empty?`, `first`

A guard follows a pattern: on an equation, or on a **`case`** alternative.

Each can be rewritten as any other.

Choose the one that says the least.

Local Definitions: `where` and `let`

The same surcharge, with its parts named in two ways:

```
surcharge :: Double -> Double
surcharge w = base + extra
  where
    base  = postage w
    extra = if w > 5000
              then 300 else 0
```

```
surcharge :: Double -> Double
surcharge w =
  let base  = postage w
      extra = if w > 5000
                then 300 else 0
  in base + extra
```

Local Definitions: `where` and `let`

The same surcharge, with its parts named in two ways:

```
surcharge :: Double -> Double
surcharge w = base + extra
  where
    base  = postage w
    extra = if w > 5000
              then 300 else 0
```

```
surcharge :: Double -> Double
surcharge w =
  let base  = postage w
      extra = if w > 5000
                then 300 else 0
  in base + extra
```

A **`where`** block hangs off an equation.
A **`let`** is an expression.

Local Definitions: `where` and `let`

The same surcharge, with its parts named in two ways:

```
surcharge :: Double -> Double
surcharge w = base + extra
  where
    base  = postage w
    extra = if w > 5000
              then 300 else 0
```

```
surcharge :: Double -> Double
surcharge w =
  let base  = postage w
      extra = if w > 5000
                then 300 else 0
  in base + extra
```

A **`where`** block hangs off an equation.
A **`let`** is an expression.

The layout matters: the local definitions start in the same column.

Local Definitions: `where` and `let`

The same surcharge, with its parts named in two ways:

```
surcharge :: Double -> Double
surcharge w = base + extra
  where
    base  = postage w
    extra = if w > 5000
              then 300 else 0
```

```
surcharge :: Double -> Double
surcharge w =
  let base  = postage w
      extra = if w > 5000
                then 300 else 0
  in base + extra
```

A **`where`** block hangs off an equation.
A **`let`** is an expression.

The layout matters: the local definitions start in the same column.

Racket's **`let`** is the second form, with indentation for parentheses.

Question: Which `w`?



```haskell
w :: Double
w = 100

heavier :: Double -> Double
heavier w = w + 1000

heavierStill :: Double
heavierStill = heavier w
```

Question: Which `w`?



```haskell
w :: Double
w = 100

heavier :: Double -> Double
heavier w = w + 1000

heavierStill :: Double
heavierStill = heavier w
```

`heavierStill` is `1100.0`.

Question: Which `w`?



```haskell
w :: Double
w = 100

heavier :: Double -> Double
heavier w = w + 1000

heavierStill :: Double
heavierStill = heavier w
```

`heavierStill` is `1100.0`.

Inside `heavier`, the parameter is the nearest binding of `w`.
The last line is outside, so its `w` is the top-level `100`.

Question: Which `w`?

```haskell
w :: Double
w = 100

heavier :: Double -> Double
heavier w = w + 1000

heavierStill :: Double
heavierStill = heavier w
```

`heavierStill` is `1100.0`.

Inside `heavier`, the parameter is the nearest binding of `w`.
The last line is outside, so its `w` is the top-level `100`.

The nearest enclosing binding wins, as it did last week.

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

```
lateFee 2
```

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

```
lateFee 2
= 2 * lateFee 1                    the second equation, n is 2
```

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

```
lateFee 2
= 2 * lateFee 1                  the second equation, n is 2
= 2 * (2 * lateFee 0)            the second equation, n is 1
```

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

```
lateFee 2
= 2 * lateFee 1                    the second equation, n is 2
= 2 * (2 * lateFee 0)              the second equation, n is 1
= 2 * (2 * 100)                    the first equation
```

Recursion

There is no loop, so a function that repeats calls itself:

```
lateFee :: Int -> Double
lateFee 0 = 100
lateFee n = 2 * lateFee (n - 1)
```

The base case comes first.
The second equation calls the function on `n - 1`. What is `lateFee 2`?

```
lateFee 2
= 2 * lateFee 1                 the second equation, n is 2
= 2 * (2 * lateFee 0)           the second equation, n is 1
= 2 * (2 * 100)                 the first equation
= 400
```

The substitution model of topic 2, one equation per step.

Task: Three Ways

Members get 10%, or 20% from ten items, and others get 10% from ten items:

```haskell
discount :: Bool -> Int -> Double
discount member n =
  if member then (if n >= 10 then 0.2 else 0.1)
  else (if n >= 10 then 0.1 else 0)
```

Two minutes, on paper

Rewrite `discount` with guards.
Then rewrite it as a `case` on `member`, with guards inside.

Task: Three Ways

Members get 10%, or 20% from ten items, and others get 10% from ten items:

```
discount :: Bool -> Int -> Double
discount member n =
  if member then (if n >= 10 then 0.2 else 0.1)
  else (if n >= 10 then 0.1 else 0)
```

Two minutes, on paper

Rewrite `discount` with guards.
Then rewrite it as a `case` on `member`, with guards inside.

Hint

A guard may combine two conditions with `&&`.

Task: Three Ways, Solved

guards

```
discount member n
  | member && n >= 10 = 0.2
  | member            = 0.1
  | n >= 10           = 0.1
  | otherwise         = 0
```

Task: Three Ways, Solved

guards

```
discount member n
  | member && n >= 10 = 0.2
  | member            = 0.1
  | n >= 10           = 0.1
  | otherwise         = 0
```

case

```
discount member n =
  case member of
    True  | n >= 10    -> 0.2
          | otherwise -> 0.1
    False | n >= 10    -> 0.1
          | otherwise -> 0
```

Task: Three Ways, Solved

guards

```
discount member n
  | member && n >= 10 = 0.2
  | member            = 0.1
  | n >= 10           = 0.1
  | otherwise         = 0
```

case

```
discount member n =
  case member of
    True  | n >= 10    -> 0.2
          | otherwise -> 0.1
    False | n >= 10    -> 0.1
          | otherwise -> 0
```

The guards read as the price list does.
The **`case`** keeps the two kinds of customer apart.

Task: Three Ways, Solved

guards

```
discount member n
  | member && n >= 10 = 0.2
  | member            = 0.1
  | n >= 10           = 0.1
  | otherwise         = 0
```

case

```
discount member n =
  case member of
    True  | n >= 10    -> 0.2
          | otherwise -> 0.1
    False | n >= 10    -> 0.1
          | otherwise -> 0
```

The guards read as the price list does.
The **`case`** keeps the two kinds of customer apart.

Both say the same thing as the nested **`if`**, and both are easier to check.

Types That Say Too Little

The `case` is the better design, and its `True` and `False` are its weak point.

Types That Say Too Little

The `case` is the better design, and its `True` and `False` are its weak point.

Boolean blindness

> `discount` takes a `Bool`: yes or no, but not to what.

Types That Say Too Little



The `case` is the better design, and its `True` and `False` are its weak point.

Boolean blindness

> `discount` takes a `Bool`: yes or no, but not to what.

Stringly typed

> `zone` takes a `String`: `"RU"` is no different from `"hello"`.

Types That Say Too Little

The `case` is the better design, and its `True` and `False` are its weak point.

Boolean blindness

> `discount` takes a `Bool`: yes or no, but not to what.

Stringly typed

> `zone` takes a `String`: `"RU"` is no different from `"hello"`.

Primitive obsession

> A parcel number as an `Int`: it adds to a weight without complaint.

Types That Say Too Little

The `case` is the better design, and its `True` and `False` are its weak point.

Boolean blindness

> `discount` takes a `Bool`: yes or no, but not to what.

Stringly typed

> `zone` takes a `String`: `"RU"` is no different from `"hello"`.

Primitive obsession

> A parcel number as an `Int`: it adds to a weight without complaint.

We will use the type system to cure these flaws and model the domain better.

5 min break



Lists

Lists

A list holds any number of values of *one* type:

```haskell
weights :: [Double]
weights = [250, 1200, 800, 5400, 90]
```

Lists

A list holds any number of values of *one* type:

```
weights :: [Double]
weights = [250, 1200, 800, 5400, 90]
```

A list is either empty, `[]`, or an element in front of a list, with `:`.
The brackets abbreviate it: `[250, 1200, 800]` is `250 : (1200 : (800 : []))`.

Lists

A list holds any number of values of *one* type:

```
weights :: [Double]
weights = [250, 1200, 800, 5400, 90]
```

A list is either empty, `[]`, or an element in front of a list, with `:`.
The brackets abbreviate it: `[250, 1200, 800]` is `250 : (1200 : (800 : []))`.

Racket's `empty` and `cons`, with a type.
What does `[1, "two"]` produce?

Lists

A list holds any number of values of *one* type:

```
weights :: [Double]
weights = [250, 1200, 800, 5400, 90]
```

A list is either empty, `[]`, or an element in front of a list, with `:`.
The brackets abbreviate it: `[250, 1200, 800]` is `250 : (1200 : (800 : []))`.

Racket's `empty` and `cons`, with a type.
What does `[1, "two"]` produce?

```
Parcels.hs:1:8: error: [GHC-39999]
    • No instance for ‘Num String’ arising from the literal ‘1’
    • In the expression: 1
      In the expression: [1, "two"]
```

`"two"` fixed the element type to **`String`**, and a **`String`** is not a number type.

Ranges

A range writes a list of consecutive values:

```
[1 .. 5]          -- [1,2,3,4,5]
[1, 3 .. 9]       -- [1,3,5,7,9],  in steps of 2
[10, 8 .. 0]      -- [10,8,6,4,2,0]
['a' .. 'e']      -- "abcde"
```

Ranges

A range writes a list of consecutive values:

```
[1 .. 5]          -- [1,2,3,4,5]
[1, 3 .. 9]       -- [1,3,5,7,9],  in steps of 2
[10, 8 .. 0]      -- [10,8,6,4,2,0]
['a' .. 'e']      -- "abcde"
```

Numbers, characters, and any type with a next value.
The end is included.

Ranges

A range writes a list of consecutive values:

```
[1 .. 5]          -- [1,2,3,4,5]
[1, 3 .. 9]       -- [1,3,5,7,9],  in steps of 2
[10, 8 .. 0]      -- [10,8,6,4,2,0]
['a' .. 'e']      -- "abcde"
```

Numbers, characters, and any type with a next value.
The end is included.

`[1 ..]` has no upper bound.
It is the stream `(from 1)` of topic 4. We will discuss this in topic 9.

Patterns on Lists



```haskell
report :: [Double] -> String
report []           = "no parcels"
report [w]          = "one parcel of " ++ show w
report (w : ws)     = "several, the first is " ++ show w
report (w1 : w2 : _) = "at least two"
```

Patterns on Lists

```haskell
report :: [Double] -> String
report []            = "no parcels"
report [w]           = "one parcel of " ++ show w
report (w : ws)      = "several, the first is " ++ show w
report (w1 : w2 : _) = "at least two"
```

`[]` is the empty list. `(w : ws)` names the first element and the rest.
`[w]` is a list of exactly one element, that is, `(w : [])`.

Patterns on Lists

```haskell
report :: [Double] -> String
report []            = "no parcels"
report [w]           = "one parcel of " ++ show w
report (w : ws)      = "several, the first is " ++ show w
report (w1 : w2 : _) = "at least two"
```

`[]` is the empty list. `(w : ws)` names the first element and the rest.
`[w]` is a list of exactly one element, that is, `(w : [])`.

Question

Which equation is never used?

Patterns on Lists

```
report :: [Double] -> String
report []            = "no parcels"
report [w]           = "one parcel of " ++ show w
report (w : ws)      = "several, the first is " ++ show w
report (w1 : w2 : _) = "at least two"
```

`[]` is the empty list. `(w : ws)` names the first element and the rest.
`[w]` is a list of exactly one element, that is, `(w : [])`.

Question

Which equation is never used?

The last one. The third equation already takes any list of two or more.

Not Every Case

Leave a case out, and GHC says so under `-Wall`:

```haskell
firstWeight :: [Double] -> Double
firstWeight (w : _) = w
```

```
Parcels.hs:31:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘firstWeight’:
        Patterns of type ‘[Double]’ not matched: []
```

Not Every Case

Leave a case out, and GHC says so under `-Wall`:

```
firstWeight :: [Double] -> Double
firstWeight (w : _) = w
```

```
Parcels.hs:31:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘firstWeight’:
        Patterns of type ‘[Double]’ not matched: []
```

Only a warning, so what happens on `firstWeight []`?

Not Every Case

Leave a case out, and GHC says so under `-Wall`:

```
firstWeight :: [Double] -> Double
firstWeight (w : _) = w
```

```
Parcels.hs:31:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘firstWeight’:
        Patterns of type ‘[Double]’ not matched: []
```

Only a warning, so what happens on `firstWeight []`?

```
ghci>  firstWeight []
*** Exception:  Non-exhaustive patterns in function firstWeight
```

Not Every Case

Leave a case out, and GHC says so under `-Wall`:

```
firstWeight :: [Double] -> Double
firstWeight (w : _) = w
```

```
Parcels.hs:31:1: warning: [GHC-62161] [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for ‘firstWeight’:
        Patterns of type ‘[Double]’ not matched: []
```

Only a warning, so what happens on `firstWeight []`?

```
ghci>  firstWeight []
*** Exception:  Non-exhaustive patterns in function firstWeight
```

No partial functions in this course

A function covers every shape of its input, ours or imported.

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
```

Recursion on a List

Two equations, one per shape of the list:

```haskell
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```haskell
totalWeight [250, 1200]
= 250 + totalWeight [1200]
```

`w` is `250`, `ws` is `[1200]`

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
= 250 + totalWeight [1200]          w is 250, ws is [1200]
= 250 + (1200 + totalWeight [])     w is 1200, ws is []
```

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
= 250 + totalWeight [1200]          w is 250, ws is [1200]
= 250 + (1200 + totalWeight [])     w is 1200, ws is []
= 250 + (1200 + 0)                  the first equation
```

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
= 250 + totalWeight [1200]          w is 250, ws is [1200]
= 250 + (1200 + totalWeight [])     w is 1200, ws is []
= 250 + (1200 + 0)                  the first equation
= 1450
```

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
= 250 + totalWeight [1200]                  w is 250, ws is [1200]
= 250 + (1200 + totalWeight [])             w is 1200, ws is []
= 250 + (1200 + 0)                          the first equation
= 1450
```

The empty list gives the start.
Each element is combined with the result for the rest.

Recursion on a List

Two equations, one per shape of the list:

```
totalWeight :: [Double] -> Double
totalWeight []       = 0
totalWeight (w : ws) = w + totalWeight ws
```

```
totalWeight [250, 1200]
= 250 + totalWeight [1200]                  w is 250, ws is [1200]
= 250 + (1200 + totalWeight [])             w is 1200, ws is []
= 250 + (1200 + 0)                          the first equation
= 1450
```

The empty list gives the start.
Each element is combined with the result for the rest.

This is the shape of most functions over lists.

Task: Count the Heavy Parcels



Two minutes, on paper

Write `countHeavy :: Double -> [Double] -> Int`, the number of weights above a limit.

`countHeavy 1000 [250, 1200, 800, 5400, 90]` is `2`.

Task: Count the Heavy Parcels

Two minutes, on paper

Write `countHeavy :: Double -> [Double] -> Int`, the number of weights above a limit.

`countHeavy 1000 [250, 1200, 800, 5400, 90]` is `2`.

```
countHeavy :: Double -> [Double] -> Int
countHeavy _ []       = 0
countHeavy limit (w : ws)
  | w > limit = 1 + countHeavy limit ws
  | otherwise = countHeavy limit ws
```

The limit is passed along unchanged.

The underscore ignores it in the base case.

Recursion With an Accumulator

The heaviest parcel, carried along as the list is walked:

```haskell
heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where
    go best []       = best
    go best (x : xs)
      | x > best  = go x xs
      | otherwise = go best xs
```

Recursion With an Accumulator

The heaviest parcel, carried along as the list is walked:

```haskell
heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where
    go best []       = best
    go best (x : xs)
      | x > best  = go x xs
      | otherwise = go best xs
```

`go` carries the best weight so far: the accumulator helper of topic 2.

Recursion With an Accumulator

The heaviest parcel, carried along as the list is walked:

```haskell
heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where
    go best []       = best
    go best (x : xs)
      | x > best  = go x xs
      | otherwise = go best xs
```

`go` carries the best weight so far: the accumulator helper of topic 2.

`0` for an empty list is a guess, not an answer.
The type could say that the result is optional, or that the input is never empty.

Recursion With an Accumulator

The heaviest parcel, carried along as the list is walked:

```haskell
heaviest :: [Double] -> Double
heaviest []       = 0
heaviest (w : ws) = go w ws
  where
    go best []       = best
    go best (x : xs)
      | x > best  = go x xs
      | otherwise = go best xs
```

`go` carries the best weight so far: the accumulator helper of topic 2.

`0` for an empty list is a guess, not an answer.
The type could say that the result is optional, or that the input is never empty.

Next week we will handle this better using types.

Racket Beside Haskell

Racket, topic 2

```
(define (total ws)
  (cond
    [(empty? ws) 0]
    [else (+ (first ws)
             (total (rest ws)))]))
```

Haskell

```
total :: [Double] -> Double
total []       = 0
total (w : ws) = w + total ws
```

Racket Beside Haskell

Racket, topic 2

```
(define (total ws)
  (cond
    [(empty? ws) 0]
    [else (+ (first ws)
             (total (rest ws)))]))
```

Haskell

```
total :: [Double] -> Double
total []       = 0
total (w : ws) = w + total ws
```

The same recursion.

The pattern `(w : ws)` does the work of `empty?`, `first`, and `rest`.

Racket Beside Haskell

Racket, topic 2

```
(define (total ws)
  (cond
    [(empty? ws) 0]
    [else (+ (first ws)
             (total (rest ws)))]))
```

Haskell

```
total :: [Double] -> Double
total []       = 0
total (w : ws) = w + total ws
```

The same recursion.
The pattern `(w : ws)` does the work of `empty?`, `first`, and `rest`.

The signature is new, and GHC checks each equation against it.

Partial Functions



The Prelude has `head` and `tail` for the first element of a list and for the rest.

```
ghci> head weights
250.0
ghci> head []
*** Exception: Prelude.head: empty list
```

Partial Functions



The Prelude has `head` and `tail` for the first element of a list and for the rest.

```
ghci> head weights
250.0
ghci> head []
*** Exception: Prelude.head: empty list
```

A **partial** function has no answer for some inputs.

GHC 9.10 warns at every use of `head` and `tail`.

Partial Functions

The Prelude has `head` and `tail` for the first element of a list and for the rest.

```
ghci> head weights
250.0
ghci> head []
*** Exception: Prelude.head: empty list
```

A **partial** function has no answer for some inputs.
GHC 9.10 warns at every use of `head` and `tail`.

Write the pattern

`(w : ws)` makes the empty case visible.
`head` hides it, and the crash comes later.

`map` and `filter` Came With Us



```
map postage weights          -- [250.0,400.0,400.0,900.0,250.0]
map (* 2) weights            -- [500.0,2400.0,1600.0,10800.0,180.0]
filter (> 2000) weights      -- [5400.0]
sum weights                  -- 7740.0
length weights               -- 5
```

`map` and `filter` Came With Us

```
map postage weights          -- [250.0,400.0,400.0,900.0,250.0]
map (* 2) weights            -- [500.0,2400.0,1600.0,10800.0,180.0]
filter (> 2000) weights      -- [5400.0]
sum weights                  -- 7740.0
length weights               -- 5
```

`(* 2)` and `(> 2000)` are **sections**: an operator with one argument filled in.
It is the closure `(lambda (w) (* w 2))` of last week.

`map` and `filter` Came With Us

```
map postage weights          -- [250.0,400.0,400.0,900.0,250.0]
map (* 2) weights            -- [500.0,2400.0,1600.0,10800.0,180.0]
filter (> 2000) weights      -- [5400.0]
sum weights                  -- 7740.0
length weights               -- 5
```

`(* 2)` and `(> 2000)` are **sections**: an operator with one argument filled in.
It is the closure `(lambda (w) (* w 2))` of last week.

```
map    :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
```

`map` and `filter` Came With Us

```
map postage weights          -- [250.0,400.0,400.0,900.0,250.0]
map (* 2) weights            -- [500.0,2400.0,1600.0,10800.0,180.0]
filter (> 2000) weights      -- [5400.0]
sum weights                  -- 7740.0
length weights               -- 5
```

`(* 2)` and `(> 2000)` are **sections**: an operator with one argument filled in.
It is the closure `(lambda (w) (* w 2))` of last week.

```
map    :: (a -> b) -> [a] -> [b]
filter :: (a -> Bool) -> [a] -> [a]
```

`foldl` waits for next week.

List Comprehensions

The **`for*/list`** of topic 4, in Haskell's notation:

Racket

```
(for*/list ([w weights]
            #:when (> w 2000))
  (postage w))
```

Haskell

```
[postage w | w <- weights, w > 2000]
```

List Comprehensions

The **`for*/list`** of topic 4, in Haskell's notation:

Racket

```
(for*/list ([w weights]
            #:when (> w 2000))
  (postage w))
```

Haskell

```
[postage w | w <- weights, w > 2000]
```

For each `w` in `weights` with `w > 2000`, take `postage w`.

The same as `map postage (filter (> 2000) weights)`.

List Comprehensions

The **`for*/list`** of topic 4, in Haskell's notation:

Racket

```
(for*/list ([w weights]
            #:when (> w 2000))
  (postage w))
```

Haskell

```
[postage w | w <- weights, w > 2000]
```

For each `w` in `weights` with `w > 2000`, take `postage w`.

The same as `map postage (filter (> 2000) weights)`.

```
[name | (name, w) <- parcels, w > 2000]     -- parcels, a list of pairs
[(i, j) | i <- [1 .. 3], j <- [i .. 3]]      -- two generators
-- [(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
```

A pattern takes each pair apart. A later generator runs inside the earlier one.

Strings Are Lists



```
"abc" == ['a', 'b', 'c']        -- True
length "books"                  -- 5
words "books 5400"              -- ["books","5400"]
```

Strings Are Lists

```
"abc" == ['a', 'b', 'c']        -- True
length "books"                  -- 5
words "books 5400"              -- ["books","5400"]
```

`String` is `[`**`Char`**`]`, so every list function works on it.

Strings Are Lists

```
"abc" == ['a', 'b', 'c']        -- True
length "books"                  -- 5
words "books 5400"              -- ["books","5400"]
```

String is [**Char**], so every list function works on it.

```
import Data.Char (toUpper)

shout :: String -> String
shout name = map toUpper name ++ "!"        -- "ANNA!"
```

The Prelude is imported by itself.
Other modules are imported by name, and the names we take go in parentheses.

Strings Are Lists

```
"abc" == ['a', 'b', 'c']        -- True
length "books"                  -- 5
words "books 5400"              -- ["books","5400"]
```

`String` is `[Char]`, so every list function works on it.

```
import Data.Char (toUpper)

shout :: String -> String
shout name = map toUpper name ++ "!"        -- "ANNA!"
```

The Prelude is imported by itself.
Other modules are imported by name, and the names we take go in parentheses.

`https://hoogle.haskell.org/` finds where a function lives.

Types of Our Own

Type Aliases

A **type alias** is a new name for a type that exists:

```
type Grams  = Double
type Rubles = Double
type Parcel = (String, Grams)

postage :: Grams -> Rubles
```

Type Aliases

A **type alias** is a new name for a type that exists:

```
type Grams  = Double
type Rubles = Double
type Parcel = (String, Grams)

postage :: Grams -> Rubles
```

The signature says which number is which.
Does this compile?

```
twice :: Rubles
twice = postage (postage 250)
```

Type Aliases

A **type alias** is a new name for a type that exists:

```
type Grams  = Double
type Rubles = Double
type Parcel = (String, Grams)

postage :: Grams -> Rubles
```

The signature says which number is which.
Does this compile?

```
twice :: Rubles
twice = postage (postage 250)
```

Yes, since GHC sees `Double` on both sides and checks nothing new.
A price went in where a weight was meant, and nothing complained.

Wrapper Types

A **wrapper** makes a type that is new to the compiler:

```haskell
newtype Grams  = Grams Double
newtype Rubles = Rubles Double

postage :: Grams -> Rubles
postage (Grams w)
  | w <= 500  = Rubles 250
  | otherwise = Rubles 900

fee :: Rubles
fee = postage (Grams 250)          -- Rubles 250
```

Wrapper Types

A **wrapper** makes a type that is new to the compiler:

```haskell
newtype Grams  = Grams Double
newtype Rubles = Rubles Double

postage :: Grams -> Rubles
postage (Grams w)
  | w <= 500  = Rubles 250
  | otherwise = Rubles 900

fee :: Rubles
fee = postage (Grams 250)          -- Rubles 250
```

`Grams` on the right of its declaration is a **constructor**.

`Grams 250` wraps a number, and the pattern `(Grams w)` unwraps it.

Wrapper Types: The Same Mistake

Does this compile?

```
twice :: Rubles
twice = postage (postage (Grams 250))
```

Wrapper Types: The Same Mistake



Does this compile?

```
twice :: Rubles
twice = postage (postage (Grams 250))
```

```
Parcels.hs:52:18: error: [GHC-83865]
    • Couldn't match expected type ‘Grams’ with actual type ‘Rubles’
    • In the first argument of ‘postage’, namely ‘(postage (Grams 250))’
```

No, since the inner call gives a **`Rubles`**, and `postage` takes a **`Grams`**.

Wrapper Types: The Same Mistake



Does this compile?

```
twice :: Rubles
twice = postage (postage (Grams 250))
```

```
Parcels.hs:52:18: error: [GHC-83865]
    • Couldn't match expected type ‘Grams’ with actual type ‘Rubles’
    • In the first argument of ‘postage’, namely ‘(postage (Grams 250))’
```

No, since the inner call gives a **`Rubles`**, and `postage` takes a **`Grams`**.

A constructor in every pattern buys a check at every use.

type, newtype, and data

```haskell
type    Grams  = Double                -- a new name
newtype Grams  = Grams Double           -- one field
data    Parcel = Parcel String Grams    -- any fields
```

```haskell
type    Grams  = Double                -- a new name
newtype Grams  = Grams Double          -- one field
data    Parcel = Parcel String Grams   -- any fields
```

A `data` type with one constructor is a tuple with a name:

```haskell
weightOf :: Parcel -> Grams
weightOf (Parcel _ w) = w
```

`type`, `newtype`, and `data`

```haskell
type    Grams  = Double                -- a new name
newtype Grams  = Grams Double          -- one field
data    Parcel = Parcel String Grams   -- any fields
```

A `data` type with one constructor is a tuple with a name:

```haskell
weightOf :: Parcel -> Grams
weightOf (Parcel _ w) = w
```

The type and the constructor may share a name.
They live in different namespaces.

`type`, `newtype`, and `data`

```haskell
type    Grams  = Double                -- a new name
newtype Grams  = Grams Double          -- one field
data    Parcel = Parcel String Grams   -- any fields
```

A `data` type with one constructor is a tuple with a name:

```haskell
weightOf :: Parcel -> Grams
weightOf (Parcel _ w) = w
```

The type and the constructor may share a name.
They live in different namespaces.

One field: `newtype`. Next week: several constructors, and fields with names.

Where the Types Could Say More

Four places today where a type was too weak for what we meant:

Where the Types Could Say More

Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.

Where the Types Could Say More

Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.
2. `discount` took a `Bool`. It meant a member or a non-member.

Where the Types Could Say More

Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.
2. `discount` took a **`Bool`**. It meant a member or a non-member.
3. `zone` took a **`String`**. It meant one of a few countries.

Where the Types Could Say More



Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.
2. `discount` took a `Bool`. It meant a member or a non-member.
3. `zone` took a `String`. It meant one of a few countries.
4. `firstWeight` had no case for `[]`, and GHC could only warn.

Where the Types Could Say More



Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.
2. `discount` took a `Bool`. It meant a member or a non-member.
3. `zone` took a `String`. It meant one of a few countries.
4. `firstWeight` had no case for `[]`, and GHC could only warn.

And one we fixed: `Grams` and `Rubles` were both `Double`, until the `newtype`.

Where the Types Could Say More



Four places today where a type was too weak for what we meant:

1. `heaviest []` returned `0`. The result is optional, or the input is never empty.
2. `discount` took a `Bool`. It meant a member or a non-member.
3. `zone` took a `String`. It meant one of a few countries.
4. `firstWeight` had no case for `[]`, and GHC could only warn.

And one we fixed: `Grams` and `Rubles` were both `Double`, until the `newtype`.

Next week: types with several constructors, and `Maybe`.
Each of the four gets a type that says what we meant.

What We Can Read Now

	Racket	Haskell
a definition	`(define (f x) ...)`	`f :: A -> B` and `f x = ...`
a choice by condition	`cond`	guards, or `if`
a choice by shape	`empty?`, `first`, `rest`	the patterns `[]` and `(w : ws)`
a local name	`let`	`where`, or `let`
select and transform	`filter`, `map`, `for*/list`	`filter`, `map`, a comprehension
a record	`struct`	`data` with one constructor
a type mistake	when the line runs	before the program runs

What We Can Read Now

	Racket	Haskell
a definition	`(define (f x) ...)`	`f :: A -> B` and `f x = ...`
a choice by condition	`cond`	guards, or `if`
a choice by shape	`empty?`, `first`, `rest`	the patterns `[]` and `(w : ws)`
a local name	`let`	`where`, or `let`
select and transform	`filter`, `map`, `for*/list`	`filter`, `map`, a comprehension
a record	`struct`	`data` with one constructor
a type mistake	when the line runs	before the program runs

We write the same way as before.

The types are new, and GHC checks them.

Quick Recap

1. What does `f :: Int -> Bool` say?
2. In what order are equations, and guards, tried?
3. What are the two shapes a list can have?
4. Why does `count + weight` fail for an `Int` and a `Double`?
5. What does a `newtype` give that a `type` alias does not?

Quick Recap

1. What does `f :: Int -> Bool` say?
   `f` is a function from an `Int` to a `Bool`.
2. In what order are equations, and guards, tried?
3. What are the two shapes a list can have?
4. Why does `count + weight` fail for an `Int` and a `Double`?
5. What does a `newtype` give that a `type` alias does not?

Quick Recap

1. What does `f :: Int -> Bool` say?
   `f` is a function from an `Int` to a `Bool`.
2. In what order are equations, and guards, tried?
   From the top, and the first that fits gives the value.
3. What are the two shapes a list can have?
4. Why does `count + weight` fail for an `Int` and a `Double`?
5. What does a `newtype` give that a `type` alias does not?

Quick Recap

1. What does `f :: Int -> Bool` say?
   `f` is a function from an `Int` to a `Bool`.
2. In what order are equations, and guards, tried?
   From the top, and the first that fits gives the value.
3. What are the two shapes a list can have?
   Empty, `[]`, or an element in front of a list, `(w : ws)`.
4. Why does `count + weight` fail for an `Int` and a `Double`?
5. What does a `newtype` give that a `type` alias does not?

Quick Recap

1. What does `f :: Int -> Bool` say?
   `f` is a function from an `Int` to a `Bool`.
2. In what order are equations, and guards, tried?
   From the top, and the first that fits gives the value.
3. What are the two shapes a list can have?
   Empty, `[]`, or an element in front of a list, `(w : ws)`.
4. Why does `count + weight` fail for an `Int` and a `Double`?
   Both arguments of `+` must have the same type, and nothing converts on its own.
5. What does a `newtype` give that a `type` alias does not?

Quick Recap

1. What does `f :: Int -> Bool` say?
   `f` is a function from an `Int` to a `Bool`.
2. In what order are equations, and guards, tried?
   From the top, and the first that fits gives the value.
3. What are the two shapes a list can have?
   Empty, `[]`, or an element in front of a list, `(w : ws)`.
4. Why does `count + weight` fail for an `Int` and a `Double`?
   Both arguments of `+` must have the same type, and nothing converts on its own.
5. What does a `newtype` give that a `type` alias does not?
   A type of its own, so the compiler refuses a value of the wrong type.

Quiz for Lecture 6

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 6

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

> Say what you learned and what stayed unclear, **specifically**.
> I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend mini project 1, if your name is drawn
- start on problem set 6, the first one in Haskell

→ **Before next week: higher-order functions and datatypes**

- Read *Learn You a Haskell*, Chapters 3, 5, and 6 (Lipovača 2011): types, recursion, higher-order functions.
- Load today's module in GHCi, and run every example on the slides. Change one and see what GHC says.

Thank you!



References

- Damas, Luis and Robin Milner (1982). **"Principal type-schemes for functional programs".** In: *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '82. Albuquerque, New Mexico: Association for Computing Machinery, pp. 207–212. ISBN: 0897910656. DOI: 10.1145/582153.582176. URL: https://doi.org/10.1145/582153.582176.
- Lipovača, Miran (2011). ***Learn You a Haskell for Great Good! A Beginner's Guide*.** No Starch Press. URL: https://learnyouahaskell.com/chapters (visited on 09/27/2026).

References ii

- Marlow, Simon (2010). ***Haskell 2010 Language Report.*** URL: https://www.haskell.org/onlinereport/haskell2010/ (visited on 09/27/2026).
- The GHC Team (2024). ***GHC User's Guide, version 9.10.*** URL: https://downloads.haskell.org/ghc/9.10.3/docs/users_guide/ (visited on 09/27/2026).